

Homework 2

Q1: Maximum Likelihood Estimation

Example 1: Suppose that X is a discrete random variable with the following probability mass function: where $0 \leq \theta \leq 1$ is a parameter. The following 10 independent observations

X	0	1	2	3
$P(X)$	$2\theta/3$	$\theta/3$	$2(1-\theta)/3$	$(1-\theta)/3$

were taken from such a distribution: (3,0,2,1,3,2,1,0,2,1). What is the maximum likelihood estimate of θ .

hint: the first step is always to write out the expression of the probability to observe all the samples given the parameter.

Q2:

Example 2: Suppose $X_1, X_2, \dots, X_n$ are i.i.d. random variables with density function $f(x|\sigma) = \frac{1}{2\sigma} \exp\left(-\frac{|x|}{\sigma}\right)$, please find the maximum likelihood estimate of σ .

Q3: Use Maximal likelihood method to estimate the parameters μ and σ for the normal distribution density

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\},$$

based on a random sample set $\{X_1, X_2, \dots, X_n\}$

Q4:

Exercise 1: Let $X_1, \dots, X_n$ be an i.i.d. sample from a Poisson distribution with parameter λ , i.e.,

$$P(X = x|\lambda) = \frac{\lambda^x e^{-\lambda}}{x!}.$$

Please find the MLE of the parameter λ .

Q5: MLE applied to neural network based probabilistic regression

In probabilistic regression, the model (e.g. neural network) output is modeled as a Gaussian distribution with both a mean and a standard deviation.

Let's denote the output of the neural network as:

- $\mu(X)$: the predicted mean for the input X
- $\sigma(X)$: the predicted standard deviation for the input X

Assuming Y follows a Gaussian distribution given X :

$$Y|X \sim \mathcal{N}(\mu(X), \sigma^2(X))$$

Derive the loss function using Maximum Likelihood Estimation (MLE), which is used to maximize the probability to see data samples (X_i, Y_i) for $i = 1, 2, \dots, N$. Note that loss function is to be minimized rather than maximized.

Q6. Stacked deep decision tree for higher classification performance (AI coding allowed)

Standard decision trees/random forests have limited prediction power compared to deep neural network. But multi-layered random forests may get their performance increased by using a stacked layer of random forest layers. The structure of the idea is below. (1) describe how the multi-layer deep random forest works; (2) copy this architecture of deep random forest to codex, and ask it implement it for classification and compare its performance with the standard random forest on this dataset. <https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones> You can ask codex to download the dataset, and do 80:10:10 splitting and test the performance.

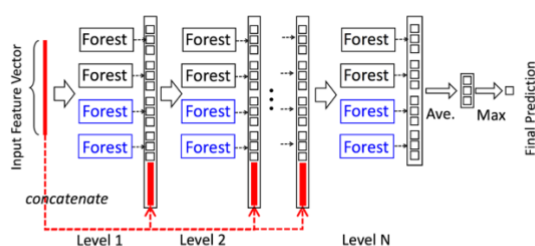


Fig. 2. Illustration of the cascade forest structure. Suppose each level of the cascade consists of two random forests (black) and two completely-random tree forests (blue). Suppose there are three classes to predict; thus, each forest will output a three-dimensional class vector, which is then concatenated for re-representation of the original input.

Q7:

For the following dataset, manually derive the decision tree using the information gain approach.

x_1	x_2	y
0	1	0
1	1	0
0	0	1
1	0	0
0	1	0
0	1	1

Q8: The following page carries examples of using logistic regression and other classifiers and calculates the performance measures.

<https://machinelearningmastery.com/get-your-hands-dirty-with-scikit-learn-now/>

Modify the regression example to implement the following function for calculating performance measures of classification:

1. Based on the predicted labels and expected labels for class 0, calculate the TP, FP, TN FN for class 0 and then to calculate accuracy, precision, recall. Your results should be the same as those got by calling `print(metrics.classification_report(expected, predicted))`
You must write your OWN implementation of these calculation. Cannot use Scikiet-learn functions.

2. Use the following statement to calculate the probability of each test sample
`probs= model.predict_proba(dataset.data)`
 and then use the scikit-learn functions to calculate AUC, and plot ROC curve.

ROC cuve example is here <http://thestatsgeek.com/2014/05/05/area-under-the-roc-curve-assessing-discrimination-in-logistic-regression/>

3. Use scikit-learn package to calculate cross-validation error, leave-one-out cv error.

You need to submit the source code of this question.

Submit all the answers and code to <http://dropbox.cse.sc.edu>